

Reviewer Pack — Metric Dimension Lower Bound on Tseitin Resolution Length

Entry 07a9b4327000 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-08 05:18:01 UTC

Metric Dimension Lower Bound on Tseitin Resolution Length

Entry ID: 07a9b4327000 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-08 05:18:01 UTC

1. Conjecture

Field A (mathematical branch): Metric Dimension Theory **Field B** (complexity object): Resolution Proof Length

Statement:

For a Tseitin formula derived from a graph G , the Resolution length is at least $2^{\Omega(\nu(G))}$, where $\nu(G)$ is the metric dimension of G . Non-expander graphs (with $\nu(G) = O(1)$) admit polynomial-length Resolution proofs.

Rationale (proposer's reasoning):

Metric dimension captures the minimum number of nodes needed to uniquely identify all others via distance. High metric dimension implies structural complexity, which correlates with exponential Resolution lower bounds for Tseitin formulas. This bridges combinatorial graph theory with proof complexity.

Taxonomy category: TSEITIN_RES (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 145bef0cb78f42f4

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.95	UNCERTAIN	SAFE
ALGEBRIZATION	SAFE	0.95	SAFE	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
---------	---------	------------	------------------	------------------

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=0, elapsed=0.3s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction
from itertools import combinations

def gaussian_elimination(Ab):
    n = len(Ab)
    for i in range(n):
        # Find pivot row
        max_row = i
        for j in range(i+1, n):
            if abs(Ab[j][i]) > abs(Ab[max_row][i]):
                max_row = j
        Ab[i], Ab[max_row] = Ab[max_row], Ab[i]

        # Eliminate below pivot
        for j in range(i+1, n):
            factor = Fraction(Ab[j][i], Ab[i][i])
            for k in range(n + 1):
                Ab[j][k] -= factor * Ab[i][k]

def solve_linear_system(A, b):
    Ab = [row[:] + [col] for row, col in zip(A, b)]
    gaussian_elimination(Ab)

    n = len(Ab)
    x = [0] * n
    for i in range(n-1, -1, -1):
        x[i] = Fraction(Ab[i][n], Ab[i][i])
        for j in range(i+1, n):
            x[i] -= Fraction(Ab[i][j] * x[j], Ab[i][i])

    return x

def metric_dimension(G):
    n = len(G)
    A = [[0] * n for _ in range(n)]
    b = [1] * n

    for i, j in combinations(range(n), 2):
        if G[i][j]:
```

```

        A[i][j] = -1
        A[j][i] = -1
        b[i] += 1
        b[j] += 1

    try:
        x = solve_linear_system(A, b)
        return sum(1 for val in x if val > 0)
    except ZeroDivisionError:
        return float('inf')

def run_trial(seed: int) -> dict:
    random.seed(seed)

    n = random.randint(5, 40)
    G = [[random.choice([0, 1]) for _ in range(n)] for _ in range(n)]
    for i in range(n):
        G[i][i] = 0

    v_G = metric_dimension(G)
    if v_G == float('inf'):
        return {
            "metric_name": "v(G)",
            "metric_value": v_G,
            "instances_tested": 1,
            "conjecture_holds": False,
            "counterexample": "mapping_undefined"
        }

    # Simulate Tseitin Resolution length (simplified)
    resolution_length = 2 ** math.ceil(v_G / math.log(n))

    return {
        "metric_name": "Resolution Length",
        "metric_value": resolution_length,
        "instances_tested": 1,
        "conjecture_holds": resolution_length >= 2 ** (math.log(n, 2) + 1),
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [random.getrandbits(32) for _ in range(30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(res["metric_value"] for res in results) / len(results)
    std_value = math.sqrt(sum((res["metric_value"] - mean_value) ** 2 for res in results) / len(results))
    support_fraction = sum(1 for res in results if res["conjecture_holds"]) / len(results)

    if all(res["conjecture_holds"] for res in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")

```

```

elif support_fraction >= 0.8:
    print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
else:
    first_failing_seed = next((seed for seed, res in zip(seeds, results) if not res["conjecture_holds"]), None)
    counterexample = "Resolution length is less than  $2^{\Omega(v(G))}$ "
    print(f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

''}
TRIAL: {'metric_name': 'Resolution Length', 'metric_value': 1, 'instances_tested': 1, 'conjecture_holds': True}
TRIAL: {'metric_name': 'Resolution Length', 'metric_value': 1, 'instances_tested': 1, 'conjecture_holds': True}
TRIAL: {'metric_name': 'Resolution Length', 'metric_value': 16, 'instances_tested': 1, 'conjecture_holds': True}
TRIAL: {'metric_name': 'Resolution Length', 'metric_value': 1, 'instances_tested': 1, 'conjecture_holds': True}
TRIAL: {'metric_name': 'Resolution Length', 'metric_value': 1, 'instances_tested': 1, 'conjecture_holds': True}
TRIAL: {'metric_name': 'v(G)', 'metric_value': inf, 'instances_tested': 1, 'conjecture_holds': False}
TRIAL: {'metric_name': 'Resolution Length', 'metric_value': 4, 'instances_tested': 1, 'conjecture_holds': True}
TRIAL: {'metric_name': 'Resolution Length', 'metric_value': 1, 'instances_tested': 1, 'conjecture_holds': True}
TRIAL: {'metric_name': 'Resolution Length', 'metric_value': 1, 'instances_tested': 1, 'conjecture_holds': True}
TRIAL: {'metric_name': 'Resolution Length', 'metric_value': 2, 'instances_tested': 1, 'conjecture_holds': True}
TRIAL: {'metric_name': 'Resolution Length', 'metric_value': 1, 'instances_tested': 1, 'conjecture_holds': True}
TRIAL: {'metric_name': 'Resolution Length', 'metric_value': 2, 'instances_tested': 1, 'conjecture_holds': True}
TRIAL: {'metric_name': 'Resolution Length', 'metric_value': 1, 'instances_tested': 1, 'conjecture_holds': True}
TRIAL: {'metric_name': 'Resolution Length', 'metric_value': 1, 'instances_tested': 1, 'conjecture_holds': True}
TRIAL: {'metric_name': 'Resolution Length', 'metric_value': 1, 'instances_tested': 1, 'conjecture_holds': True}
RESULT: FALSIFIED counterexample="Resolution length is less than  $2^{\Omega(v(G))}$ " first_failing_seed=11

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

```

{ "critic_verdict": "CHALLENGE", "reasoning": "The trials show n=1 instances tested, violating the 'n too small' failure mode. Resolution length metrics (1

```

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

```

parse_fail | next: NONE

```

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	106355
2	propose	ollama_remote	qwen3:8b	0	0	86097
3	preregistration	ollama_remote	qwen3:8b	0	0	19957
4	novelty	ollama_remote	qwen3:8b	0	0	16639
5	novelty	ollama_remote	qwen3:8b	0	0	11697

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16440
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10898
8	critic	ollama_remote	qwen3:8b	0	0	25687
9	judge	ollama_remote	qwen3:8b	0	0	16944

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 310713 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in research/audit/07a9b4327000.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/07a9b4327000.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/07a9b4327000.tar.gz (if generated)